

Specyfikacja techniczna i instrukcja implementacji systemu

Roboton_reflekton: Autonomiczny agent księgowy AI

Ewolucja systemów finansowych w kierunku pełnej autonomii wymaga integracji zaawansowanych modeli językowych, rygorystycznych struktur bazodanowych oraz niezawodnych mechanizmów komunikacyjnych. System Roboton_reflekton został zaprojektowany jako „mózg” i „ręce” ekosystemu autonomicznego księgowego, budując synergję z modułem NexusAI, który pełni rolę „oczu” i „pamięci” systemu.¹ W niniejszej specyfikacji przedstawiono szczegółowy plan budowy aplikacji, opartej na architekturze monolitu modularnego, wykorzystującej najnowocześniejsze narzędzia ekosystemu Python oraz specjalistyczne bazy danych do obsługi matematyki finansowej i dokumentacji prawnej.¹

Architektura systemu i fundamenty backendowe

Rdzeń systemu Roboton_reflekton opiera się na języku Python 3.11+, wykorzystując jego zaawansowane możliwości asynchroniczne do obsługi tysięcy jednoczesnych operacji I/O, co jest niezbędne przy masowym przetwarzaniu faktur i komunikacji z bramkami rządowymi.¹ Wybór frameworka Litestar jako fundamentu API wynika z jego wysokiej wydajności, natywnego wsparcia dla asynchroniczności oraz automatycznej generacji dokumentacji OpenAPI/Swagger, co ułatwia integrację z systemami zewnętrznymi.²

Zastosowanie wzorca Modular Monolith pozwala na zachowanie spójności bazy kodowej przy jednoczesnym logicznym odseparowaniu kluczowych modułów, takich jak *Company_creator*, *Tax_Classifier_AI* oraz *Ledger_Engine*.¹ Każdy z tych modułów posiada własne definicje danych, logikę biznesową oraz testy, komunikując się z pozostałymi poprzez wewnętrzne interfejsy lub szybną zdarzeń NATS JetStream.¹

Komponent	Technologia	Funkcja architektoniczna
Rdzeń backendu	Python 3.11+ (asyncio)	Zarządzanie logiką i orkiestracja zadań ¹
Framework API	Litestar	Obsługa żądań REST i

		generowanie specyfikacji ²
Struktura	Modular Monolith	Organizacja kodu i separacja domen ¹
Zarządzanie środowiskiem	uv (Astral)	Błyskawiczna instalacja zależności i izolacja ⁵

Wykorzystanie narzędzia uv jako menedżera środowiska i zależności drastycznie przyspiesza procesy CI/CD oraz lokalny rozwój, oferując wydajność nieosiągalną dla tradycyjnych narzędzi typu pip czy poetry.⁵ W architekturze asynchronicznej kluczowe jest stosowanie `asyncio.TaskGroup` (wprowadzonego w Python 3.11) do bezpiecznego zarządzania grupami zadań, co gwarantuje, że błąd w jednym procesie (np. pobieraniu danych z banku) nie przerwie działania całego systemu.⁶

System tożsamości i bezpieczeństwa danych

Bezpieczeństwo w systemie księgowym nie podlega negocjacjom. Roboton_reflekton implementuje wielowarstwowy model ochrony, zaczynając od autoryzacji opartej na Litestar Security i wewnętrznym mechanizmie JWT.⁷ Każdy użytkownik i agent w systemie posiada przypisane role, a dostęp do konkretnych punktów końcowych API jest weryfikowany za pomocą strażników (Guards), co pozwala na precyzyjne sterowanie uprawnieniami.⁹

Hasła użytkowników są zabezpieczone przy użyciu algorytmu Argon2id, który został uznany za standard w nowoczesnym hashowaniu haseł ze względu na swoją odporność na ataki typu brute-force wykonywane na układach GPU oraz ataki typu side-channel.¹¹ Implementacja wykorzystuje bibliotekę `argon2-cffi`, z zalecanymi parametrami zapewniającymi optymalny balans między bezpieczeństwem a wydajnością.¹¹

Parametr Argon2id	Wartość zalecana	Uzasadnienie
memory_cost	64 MB (65536 KB)	Ochrona przed optymalizacją sprzętową FPGA/ASIC ¹¹
time_cost	3 iteracje	Zapewnienie odpowiedniego czasu obliczeń ¹¹

parallelism	4 wątki	Wykorzystanie wielordzeniowości procesorów serwerowych ¹²
-------------	---------	--

Do integracji z zewnętrznymi systemami (np. systemami CRM klientów czy zewnętrznymi biurami rachunkowymi) Roboton_reflekton wykorzystuje protokół OAuth2.⁸ Zarządzanie kluczami kryptograficznymi, sekretami bazy danych oraz tokenami dostępowymi do KSeF odbywa się poprzez platformę Infisical.¹⁴ Sekrety są dodatkowo szyfrowane w spoczynku za pomocą narzędzi SOPS i Age, co chroni je przed wyciekiem nawet w przypadku nieautoryzowanego dostępu do repozytorium kodu.¹⁶

Hybrydowy model bazodanowy: TigerBeetle i PostgreSQL

Księgowość wymaga niezawodności transakcyjnej (ACID) oraz precyzji matematycznej, która wyklucza błędy zaokrągleń.¹ Dlatego Roboton_reflekton stosuje unikalne, hybrydowe podejście do przechowywania danych, łącząc bazę TigerBeetle do "matematyki finansowej" z PostgreSQL do przechowywania danych relacyjnych o klientach i metadanych dokumentów.¹

TigerBeetle to wyspecjalizowana baza danych typu OLTP, zaprojektowana wyłącznie do obsługi księgowości podwójnej (double-entry bookkeeping).¹⁷ W przeciwieństwie do ogólnych baz SQL, TigerBeetle posiada natywną wiedzę o kontach i transferach, co pozwala na przetwarzanie milionów transakcji na sekundę przy zachowaniu absolutnej spójności danych.¹⁹ Każda operacja finansowa w systemie Roboton_reflekton musi przejść przez TigerBeetle, co gwarantuje, że suma debetów zawsze odpowiada sumie kredytów.¹⁷

$$\sum \text{Debits} = \sum \text{Credits}$$

Baza danych	Typ danych	Kluczowa rola
TigerBeetle	Konta, Transfery, Sald	Gwarancja spójności bilansu i wydajność transakcyjna ¹⁷
PostgreSQL	Klienci, PKD, Metadane	Przechowywanie złożonych struktur relacyjnych i konfiguracji ¹

DuckDB	Logi, JPK, Analityka	Błyskawiczne agregacje końcomiesięczne i audyt ¹
--------	----------------------	--

Integracja z TigerBeetle odbywa się poprzez asynchronicznego klienta Python, który automatycznie grupuje (batching) wiele transferów w jedno żądanie sieciowe, co jest kluczowe dla osiągnięcia maksymalnej przepustowości.¹⁹ PostgreSQL służy jako "źródło prawdy" dla modułu *Company_creator*, przechowując parametry prawne firm, takie jak formy opodatkowania, statusy VAT oraz przypisane plany kont.¹

Komunikacja asynchroniczna i Event Bus

Roboton_reflekton nie pracuje w próżni. Jako system sterowany zdarzeniami, opiera swoją komunikację na szynie NATS JetStream.⁴ Jest to rozwiązanie łączące lekkość tradycyjnego NATS z trwałością i gwarancjami dostarczania wiadomości charakterystycznymi dla systemów takich jak Kafka.²³

Gdy system NexusAI zakończy proces ekstrakcji danych z faktury, publikuje zdarzenie `invoice.extracted` na odpowiednim subiekcie w NATS.¹ Roboton_reflekton, działając jako subskrybent, natychmiastowo przejmuje to zdarzenie i rozpoczyna cykl księgowy.¹ Wykorzystanie JetStream pozwala na implementację wzorca Event Sourcing – każde zdarzenie finansowe jest trwale zapisane w strumieniu, co umożliwia odtworzenie stanu ksiąg w dowolnym momencie.²²

Do zarządzania rozproszonymi zadaniami długotrwałymi (np. generowanie masowych deklaracji JPK czy synchronizacja z wyciągami bankowymi) system wykorzystuje bibliotekę Taskiq zintegrowaną z NATS.²⁴ Pozwala to na horyzontalne skalowanie workerów i separację warstwy obliczeniowej od API.²⁶

Parametr NATS JetStream	Konfiguracja	Cel
Retention Policy	Limits / WorkQueue	Zarządzanie przestrzenią i trwałością zadań ²²
Replay Policy	All / StartSequence	Możliwość ponownego przetworzenia historii zdarzeń ²²
Ack Policy	Explicit	Gwarancja, że żadna faktura nie zostanie pominięta ⁵

Wykorzystanie FastStream (opartego na NATS) umożliwia programowanie deklaratywne, gdzie obsługa wiadomości z kolejki przypomina definicję endpointów w nowoczesnych frameworkach webowych, co znacząco obniża próg wejścia dla programistów i ułatwia testowanie logiki asynchronicznej.²³

Przetwarzanie dokumentów i zaawansowany potok OCR

Proces wprowadzania danych do systemu Roboton_reflekton zaczyna się od warstwy OCR, która musi być nie tylko dokładna, ale i odporna na błędy interpretacji.²⁸ Zastosowano tu mechanizm walidacji krzyżowej (Cross-Validation), w którym Surya OCR pełni rolę silnika podstawowego, a PaddleOCR V4 służy jako walidator.²⁸

Surya OCR jest nowoczesnym systemem, który wykracza poza proste rozpoznawanie znaków, oferując ekstrakcję świadomą układu strony (Layout-aware extraction).²⁸ Potrafi on zidentyfikować, które fragmenty tekstu są nagłówkami, tabelami czy stopkami, co jest kluczowe dla poprawnej interpretacji pozycji faktury.²⁸ PaddleOCR V4, znany ze swojej szybkości i precyzji w rozpoznawaniu tekstu wielojęzycznego, weryfikuje wyniki Surya, szczególnie w obszarach krytycznych, takich jak NIP kontrahenta czy kwoty brutto.²⁸

Algorytm Konsensusu działa w oparciu o następujące kroki:

1. **Preprocessing OpenCV:** Obraz dokumentu jest poddawany korekcie (prostowanie, usuwanie szumów, poprawa kontrastu) w celu zmaksymalizowania czytelności.²⁸
2. **Podwójna Ekstrakcja:** Oba silniki OCR przetwarzają dokument równoległe.²⁸
3. **Porównanie Sum Kontrolnych:** System wylicza sumę netto + VAT i porównuje ją z odczytaną kwotą brutto. Jeśli oba silniki zgadzają się co do kwot, dokument przechodzi do księgowania automatycznego.²⁸
4. **Walidacja Krzyżowa:** Jeśli występuje rozbieżność, system sprawdza pola krytyczne (NIP, data, waluta). W przypadku niezgodności, dokument trafia do modułu Human-in-the-Loop.¹

Dla zapewnienia najwyższej jakości ekstrakcji, Roboton_reflekton wykorzystuje modele LayoutLMv3 oraz HerBERT z biblioteki Hugging Face.³¹ LayoutLMv3 jest szczególnie skuteczny w klasyfikowaniu elementów faktur, ponieważ analizuje jednocześnie tekst i jego położenie geometryczne na stronie.³¹ Modele te są uruchamiane lokalnie przy użyciu kwantyzacji INT8/GGUF, co pozwala na uzyskanie wysokiej wydajności na standardowych procesorach serwerowych bez konieczności posiadania potężnych układów GPU.¹

Uczenie maszynowe i Agenci: CrewAI oraz LangGraph

Sercem inteligencji Roboton_reflekton jest system Agentic AI oparty na frameworkach CrewAI lub LangGraph.¹ Zamiast polegać na pojedynczym modelu LLM, system orkiestruje zespół

wyspecjalizowanych agentów, którzy sprawdzają się nawzajem, symulując pracę biura rachunkowego.¹

W systemie zdefiniowano trzy kluczowe role:

1. **Agent Księgowy:** Analizuje pozycje faktury i na podstawie danych z NexusAI oraz profilu firmy w Company_creator proponuje dekret księgowy (przypisanie do konta Wn/Ma).¹
2. **Agent Audytor:** Weryfikuje propozycję księgowego pod kątem przepisów podatkowych (np. czy dany wydatek może być kosztem uzyskania przychodu w przypadku auta mieszanego w JDG).¹
3. **Agent Główny Księgowy:** Ostatecznie zatwierdza operację i inicjuje zapis w TigerBeetle oraz PostgreSQL.¹

Proces ten jest wspierany przez Sentence-Transformers, które służą do semantycznego wyszukiwania podobnych transakcji z przeszłości (RAG - Retrieval-Augmented Generation), co pozwala AI na naukę specyficznych nawyków księgowych danej firmy.¹ Wykorzystanie PyTorch 2.x zapewnia stabilne środowisko do uruchamiania tych modeli w trybie asynchronicznym.¹

BPM i zarządzanie przepływem pracy (Workflow)

Cykl życia faktury w Roboton_reflekton jest rygorystycznie zarządzany przez maszynę stanów implementowaną za pomocą biblioteki python-statemachine.³⁴ Pozwala to na uniknięcie błędów procesowych, takich jak zaksięgowanie tego samego dokumentu dwa razy lub wysłanie niezatwierdzonej deklaracji do urzędu.³⁴

Każdy dokument w systemie przechodzi przez predefiniowane stany:

- **INITIAL:** Faktura odebrana z NexusAI lub KSeF.
- **PROCESSING:** Trwa OCR i klasyfikacja AI.
- **PENDING_APPROVAL:** Dokument czeka na ostateczne zatwierdzenie przez użytkownika (Human-in-the-Loop).¹
- **BOOKED:** Transakcja zapisana w TigerBeetle i PostgreSQL.
- **ARCHIVED:** Dokument wysłany do archiwum długoterminowego.

Zastosowanie wzorca CQRS (Command Query Responsibility Segregation) przy wsparciu NATS JetStream pozwala na oddzielenie operacji zapisu (księgowanie) od odczytu (raportowanie i dashboardy).²³ Dzięki temu użytkownik widzi aktualny stan swoich podatków w czasie rzeczywistym, bez obciążania głównego silnika księgowego skomplikowanymi zapytaniami analitycznymi.¹

Integracje techniczne: KSeF, e-Deklaracje i Bankowość

Roboton_reflekton jest systemem "native-gov", co oznacza, że integracja z polskimi systemami administracyjnymi jest wbudowana w jego fundamenty.¹ Kluczowym elementem jest moduł KSeF, który komunikuje się z bramką Ministerstwa Finansów przy użyciu biblioteki HTTPX oraz

modeli danych Pydantic V2.³⁷

System automatycznie obsługuje strukturę logiczną FA(3), pobierając faktury ustrukturyzowane w formacie XML i przekazując je bezpośrednio do silnika *Tax_Classifier_AI*, co eliminuje błędy OCR (dokumenty z KSeF są źródłem prawdy).¹ Do generowania deklaracji podatkowych (JPK_V7, PIT, CIT) wykorzystywana jest biblioteka xsdata, która gwarantuje zgodność generowanych plików XML ze schematami XSD Ministerstwa Finansów.⁴⁰

Integracja	Protokół / Biblioteka	Cel biznesowy
KSeF	HTTPX / Pydantic V2	Automatyczne pobieranie i wysyłanie faktur ³⁷
e-Urząd	xsdata / HTTPX	Wysyłka deklaracji JPK i PIT/CIT ⁴⁰
GUS BIR / VIES	SOAP / REST	Automatyczna walidacja kontrahentów po NIP ⁴²
PSD2	Authlib / HTTPX	Pobieranie wyciągów bankowych i parowanie płatności ³⁶

Ważnym aspektem integracji jest mechanizm **Circuit Breaker**, który chroni system przed awariami zewnętrznych API.⁴⁴ Jeśli serwery Ministerstwa Finansów są przeciążone lub niedostępne, Roboton_reflekton automatycznie kolejkuje zadania i ponawia je po określonym czasie, informując użytkownika o statusie połączenia.⁴⁴

Frontend i interfejs użytkownika: Flet (Flutter for Python)

Interfejs użytkownika systemu Roboton_reflekton został zbudowany w technologii Flet, która umożliwia tworzenie natywnych aplikacji desktopowych i mobilnych przy użyciu czystego Pythona.¹ Flet bazuje na silniku Flutter, co zapewnia wysoką wydajność renderowania oraz nowoczesny wygląd (Material Design 3) na wszystkich platformach.

Użytkownik ma dostęp do:

- **Dashboardu podatkowego:** Widok w czasie rzeczywistym przewidywanych podatków (VAT, PIT/CIT) i składek ZUS na 20. dzień miesiąca.¹
- **Repozytorium dokumentów:** Przeglądanie faktur zNexusAI z możliwością podglądu

wyników OCR i decyzji podjętych przez AI.¹

- **Modułu weryfikacji:** Interfejs do szybkiego zatwierdzania propozycji księgowych (Human-in-the-Loop).¹

Integracja frontendu z asynchronicznym backendem odbywa się poprzez WebSockets (Litestar), co pozwala na natychmiastowe powiadamianie użytkownika o zakończeniu procesów, takich jak odebranie nowej faktury z KSeF czy błąd walidacji danych.²

Przechowywanie danych i wersjonowanie dokumentów

System wykorzystuje Native File System zarządzany przez bibliotekę fsspec (Filesystem Spec).⁴⁸ fsspec zapewnia ujednolicony interfejs do różnych typów pamięci masowej, co pozwala na płynne przełączanie się między lokalnym dyskiem a chmurą (np. Cloudflare R2 czy MinIO) bez zmiany kodu aplikacji.⁴⁹

Lokalny magazyn dokumentów implementuje mechanizm wersjonowania, co jest kluczowe w audycie księgowym.¹ Każda zmiana w dokumencie lub jego metadanych tworzy nową wersję pliku, umożliwiając śledzenie historii modyfikacji. Wykorzystanie fsspec pozwala również na transparentne cachowanie dokumentów z chmury na lokalnym serwerze w celu przyspieszenia pracy AI.⁵²

Funkcja Storage	Implementacja	Korzyść
Abstrakcja	fsspec	Niezależność od dostawcy chmury ⁴⁸
VFS	DirFileSystem	Bezpieczna izolacja katalogów firmowych ⁵³
Buforowanie	fsspec.core.OpenFile	Szybki dostęp do często używanych faktur ⁵¹
Importowanie	fsspec-python	Możliwość importowania modułów kodu z chmury ⁵⁴

Orkiestracja, konteneryzacja i wdrożenie (Self-hosting)

Roboton_reflekton jest systemem zaprojektowanym do działania w trybie self-hosted, co gwarantuje przedsiębiorcy pełną suwerenność nad jego danymi finansowymi.¹ Wdrożenie odbywa się przy użyciu narzędzia Pulumi (Python SDK), które pozwala na definiowanie infrastruktury jako kodu (IaC).⁶

Do konteneryzacji wykorzystywany jest Podman – nowoczesna, bezpieczna alternatywa dla Dockera, działająca bez demona (daemonless) i w trybie rootless, co drastycznie ogranicza wektory ataku na serwer.⁵⁶ Kluczowe komponenty backendu są kompilowane do plików wykonywalnych za pomocą Nuitka, co zwiększa wydajność kodu Pythona i chroni własność intelektualną algorytmów AI.¹

Zarządzanie wdrożeniami ułatwia Coolify – otwarta platforma typu PaaS (Platform as a Service), która automatyzuje budowanie obrazów, zarządzanie bazami danych i monitorowanie kontenerów.⁵⁸ W celu zapewnienia bezpiecznego dostępu do serwera z zewnątrz (bez publicznego IP), system wykorzystuje:

1. **Headscale:** Prywatna implementacja Tailscale, tworząca sieć VPN typu Mesh.¹
2. **Rathole / frp:** Narzędzia do ekspozycji usług przez NAT, charakteryzujące się wysoką wydajnością i niskim zużyciem zasobów.⁵⁹

Narzędzie	Wydajność	Bezpieczeństwo	Zastosowanie
Rathole	Wysoka (Rust)	Noise Protocol / TLS	Tunelowanie ruchu przez NAT ⁵⁹
frp	Średnia (Go)	Szyfrowanie sesji	Alternatywa z dashboardem ⁶¹
Podman	Wysoka	Rootless containers	Izolacja usług systemowych ⁵⁶

Monitoring, Logowanie i SIEM

Niezawodność systemu Roboton_reflekton jest monitorowana przez zintegrowany stos SIEM (Security Information and Event Management).¹ Logowanie realizowane jest przez bibliotekę loguru, która zapewnia strukturalne i czytelne logi, przesyłane następnie do narzędzia Vector.⁶³

Vector agreguje logi i metryki, przesyłając je do dwóch miejsc:

1. **VictoriaMetrics:** Wysokowydajna baza danych szeregów czasowych (Time Series Database), która przechowuje metryki wydajnościowe systemu i wskaźniki biznesowe

(np. czas przetwarzania faktury).¹

2. **DuckDB:** Wykorzystywany do zaawansowanej analizy logów przy użyciu SQL.⁶⁴ Dzięki formatowi kolumnowemu, DuckDB potrafi w ułamku sekundy przeszukać gigabajty logów w poszukiwaniu anomalii bezpieczeństwa czy błędów w cyklach księgowych.⁶⁵

Do monitorowania błędów aplikacyjnych i wyjątków w czasie rzeczywistym służy Sentry Lite, zintegrowane natywnie z frameworkiem Litestar.⁶⁷ Zapewnia to programistom pełny wgląd w błędy wraz ze stosami wywołań (stack trace) i kontekstem zdarzenia.⁶⁷

Backup i Disaster Recovery

Strategia ochrony danych w Roboton_reflekton opiera się na zasadzie "zero utraty danych".¹⁷ System wykorzystuje Litestream do ciągłej replikacji baz danych SQLite (używanych w niektórych modułach NexusAI) do obiektowej pamięci masowej (Cloudflare R2 lub MinIO).⁶⁸ Litestream przechwytuje zmiany na poziomie logu zapisu (WAL) i natychmiastowo przesyła je do chmury, co pozwala na przywrócenie bazy do dowolnego punktu w czasie (Point-in-Time Recovery).⁶⁸

Dla pozostałych danych (PostgreSQL, pliki dokumentów) stosowane jest narzędzie Kopia, które wykonuje deduplikowane i szyfrowane kopie zapasowe.⁶⁸ Dzięki deduplikacji, wielokrotne wersje tego samego dokumentu zajmują minimalną ilość miejsca na zdalnym serwerze backupu.⁵⁸

Strategia testowania i QA

Zapewnienie poprawności rozliczeń podatkowych wymaga testowania wykraczającego poza standardowe Unit Testy. Roboton_reflekton wdraża kompleksowy model testowy:

- **Pytest + Hypothesis:** Testy bazujące na właściwościach (Property-based testing), które generują tysiące kombinacji danych wejściowych w celu znalezienia błędów w logice podatkowej, których programista mógłby nie przewidzieć.⁷¹
- **Playwright (Python):** Automatyczne testy E2E (End-to-End) interfejsu Flet, sprawdzające kompletne ścieżki użytkownika od wgrania PDF po wygenerowanie JPK.¹
- **Schemathesis:** Testowanie API Litestar pod kątem zgodności ze specyfikacją OpenAPI, automatycznie wykrywające błędy w walidacji danych wejściowych i niespójności w odpowiedziach serwera.⁷²

Każda zmiana w kodzie jest weryfikowana w Woodpecker CI (Self-hosted runner), co gwarantuje, że do produkcji trafia wyłącznie przetestowany i bezpieczny kod.⁷⁴

Narzędzia ETL i analityka danych

W celu przygotowania raportów końcomiesięcznych oraz analizy rentowności firmy, Roboton_reflekton wykorzystuje nowoczesne narzędzia przetwarzania danych.¹ Biblioteka dlt

(data load tool) służy do budowania potoków ETL, które pobierają dane z różnych źródeł (TigerBeetle, PostgreSQL, bankowość) i ładują je do analitycznej bazy DuckDB.⁶⁶

Do szybkiego przetwarzania dużych ramek danych (np. tysiące pozycji z faktur zakupowych) wykorzystywany jest Polars – najszybsza obecnie biblioteka DataFrame dla Pythona, napisana w Rust.¹ Polars umożliwia wykonywanie skomplikowanych agregacji finansowych w czasie rzeczywistym, co jest niezbędne dla funkcji doradczych systemu (np. symulacja zmiany formy opodatkowania).¹

Implementacja końcowa: Instrukcja dla Manus AI

Aby zbudować system Roboton_reflekton zgodnie z powyższą specyfikacją, Manus AI musi postępować według następującej ścieżki krytycznej:

1. **Inicjalizacja środowiska:** Skonfiguruj projekt przy użyciu uv, instalując Litestar, TigerBeetle-Python oraz NATS-py.⁴
2. **Budowa rdzenia finansowego:** Zdefiniuj konta i transfery w TigerBeetle, zapewniając atomowość operacji księgowych.¹⁷
3. **Implementacja Company_creator:** Stwórz silnik reguł w PostgreSQL, implementując wzorzec Strategy dla polskich form opodatkowania.¹
4. **Konfiguracja potoku OCR:** Zintegruj Surya OCR i PaddleOCR z walidacją konsensusu w OpenCV.²⁸
5. **Orkiestracja Agentów AI:** Wdroż CrewAI do obsługi ról księgowego i audytora, wykorzystując LayoutLMv3 do analizy faktur.¹
6. **Uruchomienie Event Bus:** Skonfiguruj NATS JetStream jako trwały strumień zdarzeń finansowych.²²
7. **Integracje rządowe:** Zaimplementuj asynchronicznego klienta KSeF i JPK przy użyciu HTTPX i xsdata.³⁷
8. **Wdrożenie i bezpieczeństwo:** Skonfiguruj Podman i Pulumi do lokalnej orkiestracji oraz Infisical do zarządzania sekretami.¹⁴

Roboton_reflekton, jako autonomiczny agent księgowy, zmienia rolę księgowego z "wprowadzacza danych" na "nadzorcę AI", eliminując 90% ręcznej pracy przy zachowaniu pełnej zgodności z Ustawą o Rachunkowości.¹ System ten stanowi gotowe rozwiązanie dla firm dążących do cyfrowej transformacji swoich finansów w oparciu o najbezpieczniejsze i najwydajniejsze technologie dostępne na rynku.¹

Cytowane prace

1. Budowa autonomicznego księgowego AI.pdf
2. Litestar library documentation — Litestar Framework, otwierano: kwietnia 22, 2026, <https://docs.litestar.dev/>
3. Configuring schema generation - OpenAPI - Litestar Docs, otwierano: kwietnia 22, 2026, https://docs.litestar.dev/main/usage/openapi/schema_generation.html

4. How to Use NATS with Python - OneUptime, otwierano: kwiecień 22, 2026, <https://oneuptime.com/blog/post/2026-02-02-nats-python/view>
5. NatsBroker - FastStream - airt, otwierano: kwiecień 22, 2026, https://dummy.faststream.airt.ai/latest/public_api/faststream/nats/NatsBroker/
6. Python | Languages & SDKs | Pulumi Docs, otwierano: kwiecień 22, 2026, <https://www.pulumi.com/docs/iac/languages-sdks/python/>
7. jwt — Litestar Framework, otwierano: kwiecień 22, 2026, <https://docs.litestar.dev/2/reference/security/jwt.html>
8. JWT Security Backends - Litestar Framework - Litestar Docs, otwierano: kwiecień 22, 2026, <https://docs.litestar.dev/main/usage/security/jwt.html>
9. jwt - Litestar Framework, otwierano: kwiecień 22, 2026, <https://docs.litestar.dev/main/reference/security/jwt.html>
10. Building with Litestar and AI Agents : r/Python - Reddit, otwierano: kwiecień 22, 2026, https://www.reddit.com/r/Python/comments/1new8g8/building_with_litestar_and_ai_agents/
11. How to Hash a Password - argon2-cffi 25.1.0 documentation, otwierano: kwiecień 22, 2026, <https://argon2-cffi.readthedocs.io/en/stable/howto.html>
12. Password Hashing: Scrypt, Bcrypt and ARGON2 | by Michele Prezioso | Medium, otwierano: kwiecień 22, 2026, <https://medium.com/@mprezioso/password-hashing-pbkdf2-scrypt-bcrypt-and-argon2-e25aaf41598e>
13. Argon2 Password Hashing Parameter Security - Stack Overflow, otwierano: kwiecień 22, 2026, <https://stackoverflow.com/questions/67697188/argon2-password-hashing-parameter-security>
14. SDKs - Infisical, otwierano: kwiecień 22, 2026, <https://infisical.com/docs/sdks/overview>
15. Client Ecosystem - Infisical, otwierano: kwiecień 22, 2026, <https://infisical.com/docs/documentation/getting-started/concepts/client-integrations>
16. Python - Infisical, otwierano: kwiecień 22, 2026, <https://infisical.com/docs/documentation/guides/python>
17. Start - TigerBeetle Docs, otwierano: kwiecień 22, 2026, <https://docs.tigerbeetle.com/start/>
18. TigerBeetle Docs, otwierano: kwiecień 22, 2026, <https://docs.tigerbeetle.com/single-page/>
19. tigerbeetle-python, otwierano: kwiecień 22, 2026, <https://docs.tigerbeetle.com/coding/clients/python/>
20. Requests - TigerBeetle Docs, otwierano: kwiecień 22, 2026, <https://docs.tigerbeetle.com/coding/requests/>
21. In-Process Analytical Data Management with DuckDB - InfoQ, otwierano: kwiecień 22, 2026, <https://www.infoq.com/articles/analytical-data-management-duckdb/>
22. JetStream - NATS Docs, otwierano: kwiecień 22, 2026, <https://docs.nats.io/nats-concepts/jetstream>

23. NATS JetStream - FastStream, otwierano: kwiecień 22, 2026,
<https://faststream.ag2.ai/latest/nats/jetstream/>
24. taskiq-nats/README.md at master - GitHub, otwierano: kwiecień 22, 2026,
<https://github.com/taskiq-python/taskiq-nats/blob/master/README.md>
25. taskiq-python/taskiq-faststream: FastStream - Taskiq integration to provide you with a great scheduling feature - GitHub, otwierano: kwiecień 22, 2026,
<https://github.com/taskiq-python/taskiq-faststream>
26. taskiq - PyPI, otwierano: kwiecień 22, 2026, <https://pypi.org/project/taskiq/>
27. JetStream - NATS Docs, otwierano: kwiecień 22, 2026,
https://docs.nats.io/using-nats/developer/develop_jetstream
28. Surya OCR: A Practical Deep Dive into Multilingual OCR for Real ..., otwierano: kwiecień 22, 2026,
<https://medium.com/@programmerjay1215/surya-ocr-a-practical-deep-dive-into-multilingual-ocr-for-real-world-documents-e2ab79363ddb>
29. Compact Multimodal Language Models as Robust OCR Alternatives for Noisy Textual Clinical Reports - arXiv, otwierano: kwiecień 22, 2026,
<https://arxiv.org/html/2511.13523v1>
30. 8 Top Open-Source OCR Models Compared: A Complete Guide - Modal, otwierano: kwiecień 22, 2026,
<https://modal.com/blog/8-top-open-source-ocr-models-compared>
31. Theivaprakasham/layoutlmv3-finetuned-invoice - Hugging Face, otwierano: kwiecień 22, 2026,
<https://huggingface.co/Theivaprakasham/layoutlmv3-finetuned-invoice>
32. Models - Hugging Face, otwierano: kwiecień 22, 2026,
<https://huggingface.co/models?sort=downloads&search=layout+invoice>
33. LayoutLMv3 - Hugging Face, otwierano: kwiecień 22, 2026,
https://huggingface.co/docs/transformers/model_doc/layoutlmv3
34. python-statemachine - PyPI, otwierano: kwiecień 22, 2026,
<https://pypi.org/project/python-statemachine/>
35. Conditions and Validators - python-statemachine 2.4.0, otwierano: kwiecień 22, 2026, <https://python-statemachine.readthedocs.io/en/v2.4.0/guards.html>
36. python-statemachine 3.0.0, otwierano: kwiecień 22, 2026,
<https://python-statemachine.readthedocs.io/en/latest/index.html>
37. Poland KSeF 2.0 API & FA(3) Schema Published 2025 - SNI Technology, otwierano: kwiecień 22, 2026,
<https://snitechnology.net/poland-ksef-20-api-and-fa3-schema-published-2025/>
38. Understanding Poland's KSeF 2.0 API Documentation and FA(3) Structure - RTC Suite, otwierano: kwiecień 22, 2026,
<https://rtcsuite.com/understanding-polands-ksef-2-0-api-documentation-and-fa3-structure-key-changes-and-released-api-documentation/>
39. Poland: KSeF 2.0 Official API Documentation and FA(3) Logical Structure Published | Sovos, otwierano: kwiecień 22, 2026,
<https://sovos.com/regulatory-updates/vat/poland-ksef-2-0-official-api-documentation-and-fa3-logical-structure-published/>
40. Generate files to be signed and uploaded - Data Exchange component,

- otwierano: kwiecień 22, 2026, <https://kwd.mf.gov.pl/client>
41. Contracting in Poland: A Guide for International Contractors, otwierano: kwiecień 22, 2026, <https://contractortaxation.com/contracting-in-poland-a-guide-for-international-contractors/>
 42. Setting up Independent Contractor & freelancer taxes in Poland (2025) - Abillio, otwierano: kwiecień 22, 2026, <https://abill.io/en/blog/setting-up-independent-contractor-freelancer-taxes-in-poland/>
 43. How to set up as an independent contractor in Poland - Remote, otwierano: kwiecień 22, 2026, <https://remote.com/blog/contractor-management/set-up-as-independent-contractor-poland>
 44. How to Implement Circuit Breakers in Python - OneUptime, otwierano: kwiecień 22, 2026, <https://oneuptime.com/blog/post/2026-01-23-python-circuit-breakers/view>
 45. Circuit Breaker Pattern - Long Bui's Blog, otwierano: kwiecień 22, 2026, <https://blog.longbui.net/circuit-breaker-pattern>
 46. marshallmcdonnell/python-circuit-breaker-examples - GitHub, otwierano: kwiecień 22, 2026, <https://github.com/marshallmcdonnell/python-circuit-breaker-examples>
 47. README.md - community-of-python/circuit-breaker-box - GitHub, otwierano: kwiecień 22, 2026, <https://github.com/community-of-python/circuit-breaker-box/blob/main/README.md>
 48. fsspec: Filesystem interfaces for Python — fsspec 2025.12.0.post4+g51411a04a.d20251217 documentation, otwierano: kwiecień 22, 2026, <https://filesystem-spec.readthedocs.io/>
 49. Announcing filesystem-spec - Martin Durant, otwierano: kwiecień 22, 2026, <https://martindurant.github.io/blog/filesystem-spec-released/>
 50. Usage — fsspec 2025.12.0.post0+gf199984f4.d20251203 documentation, otwierano: kwiecień 22, 2026, <https://filesystem-spec.readthedocs.io/en/stable/usage.html>
 51. Features of fsspec — fsspec 2025.12.0.post0+gf199984f4 ..., otwierano: kwiecień 22, 2026, <https://filesystem-spec.readthedocs.io/en/stable/features.html>
 52. Introducing Remote Content Caching with FSSpec - Anaconda, otwierano: kwiecień 22, 2026, <https://www.anaconda.com/blog/fsspec-remote-caching>
 53. Initialize fsspec DirFileSystem from a URL - python - Stack Overflow, otwierano: kwiecień 22, 2026, <https://stackoverflow.com/questions/75630145/initialize-fsspec-dirfilesystem-from-a-url>
 54. fsspec-python - PyPI, otwierano: kwiecień 22, 2026, <https://pypi.org/project/fsspec-python/>
 55. Self-hosted alternative to frp/ngrok, with better performance and lower memory consumption : r/selfhosted - Reddit, otwierano: kwiecień 22, 2026,

- https://www.reddit.com/r/selfhosted/comments/rvutvq/selfhosted_alternative_to_frpngrok_with_better/
56. Dynamic Resource Providers | Pulumi Docs, otwierano: kwiecień 22, 2026, <https://www.pulumi.com/docs/iac/concepts/providers/dynamic-providers/>
 57. Exploring the Power of Pulumi Dynamic Providers — a Practical ..., otwierano: kwiecień 22, 2026, <https://medium.com/@imunscarred/exploring-the-power-of-pulumi-dynamic-providers-a-practical-example-in-python-aac002db2c78>
 58. SQLite backups with Litestream · coolabsio coolify · Discussion #6702 - GitHub, otwierano: kwiecień 22, 2026, <https://github.com/coolabsio/coolify/discussions/6702>
 59. Using Rathole – Reverse proxy for NAT traversal on Linux - ComputingForGeeks, otwierano: kwiecień 22, 2026, <https://computingforgeeks.com/using-rathole-reverse-proxy-for-nat-traversal-on-linux/>
 60. GitHub - rathole-org/rathole: A lightweight and high-performance reverse proxy for NAT traversal, written in Rust. An alternative to frp and ngrok., otwierano: kwiecień 22, 2026, <https://github.com/rathole-org/rathole>
 61. FRP vs. Rathole vs. ngrok - What's the Best Reverse Tunneling Solution? - xTom, otwierano: kwiecień 22, 2026, <https://xtom.com/blog/frp-rathole-ngrok-comparison-best-reverse-tunneling-solution/>
 62. rathole - crates.io: Rust Package Registry, otwierano: kwiecień 22, 2026, <https://crates.io/crates/rathole/0.2.0>
 63. Vector output sink for log forwarding - New Relic Documentation, otwierano: kwiecień 22, 2026, <https://docs.newrelic.com/docs/logs/forward-logs/vector-output-sink-log-forwarding/>
 64. Logging - DuckDB, otwierano: kwiecień 22, 2026, https://duckdb.org/docs/current/operations_manual/logging/overview
 65. DuckDB for Data Engineers: Speed Up Your Data Pipelines 10x and More - DataCamp, otwierano: kwiecień 22, 2026, <https://www.datacamp.com/tutorial/duckdb-to-speed-up-data-pipelines>
 66. The Data Engineer's Guide to Efficient Log Parsing with DuckDB/MotherDuck, otwierano: kwiecień 22, 2026, <https://motherduck.com/blog/json-log-analysis-duckdb-motherduck/>
 67. Python SDK Improved Integrations Support | Sentry Changelog, otwierano: kwiecień 22, 2026, <https://sentry.io/changelog/python-sdk-improved-integrations-support/>
 68. Database Backups - Dubly - Mintlify, otwierano: kwiecień 22, 2026, <https://www.mintlify.com/scmmishra/dubly/deployment/backups>
 69. Going Production-Ready with SQLite: How Litestream Makes It Possible | by cosmicray001, otwierano: kwiecień 22, 2026, <https://medium.com/@cosmicray001/going-production-ready-with-sqlite-how-litestream-makes-it-possible-74f894fc96f0>

70. Replicating Directories - Litestream, otwierano: kwietnia 22, 2026,
<https://litestream.io/guides/directory/>
71. Understanding Data Generation - Schemathesis - Read the Docs, otwierano:
kwietnia 22, 2026,
<https://schemathesis.readthedocs.io/en/stable/explanations/data-generation/>
72. Quick Start Guide - Schemathesis - Read the Docs, otwierano: kwietnia 22, 2026,
<https://schemathesis.readthedocs.io/en/stable/quick-start/>
73. Testing Python Apps - Schemathesis, otwierano: kwietnia 22, 2026,
<https://schemathesis.readthedocs.io/en/stable/guides/python-apps/>
74. Workflow syntax - Woodpecker CI, otwierano: kwietnia 22, 2026,
<https://woodpecker-ci.org/docs/usage/workflow-syntax>
75. Your first pipeline - Woodpecker CI, otwierano: kwietnia 22, 2026,
<https://woodpecker-ci.org/docs/usage/intro>
76. Starlark-Powered Pipelines in Woodpecker CI - DEV Community, otwierano:
kwietnia 22, 2026,
<https://dev.to/jankaritech/starlark-powered-pipelines-in-woodpecker-ci-3ogf>
77. How to Create Your Own Vector Data Warehouse with Ducklake: Part 1 — Build -
Medium, otwierano: kwietnia 22, 2026,
<https://medium.com/towardsdev/how-to-create-your-own-vector-data-warehouse-with-ducklake-part-1-build-b30069a62062>